



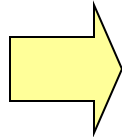
UNIVERSITY OF SCIENCE
HO CHI MINH CITY

Configuration Management

Objectives

- Explain the importance of software configuration management (SCM)
- Describe key SCM activities
 - SCM planning
 - Change management
 - Version management
 - System building
- Introduce popular SCM tools

Topics covered

- 
- Configuration management planning
 - Change management
 - Version and release management
 - System building
 - SCM tools

Configuration management

- New versions of software systems are created as they change
 - ❑ For different machines/platforms
 - ❑ Offering different functionality
 - ❑ Tailored for particular user requirements

- CM is concerned with managing evolving software systems
 - ❑ System change is a team activity
 - ❑ CM aims to control problems involved in making changes

Configuration management

- Involves application of procedures and standards to manage evolving software
- Usually part of quality management processes
- CM involves
 - Identifying changing work products
 - Establishing relationships among changes
 - Defining mechanism to manage the changes
 - Version control, tracking changes, traceability, parallel development
 - Controlling, modifying, and reporting changes

CM standards

- CM is based on a set of standards
- Standards define
 - ❑ how items are identified
 - ❑ how changes are controlled
 - ❑ how new versions are managed
- Example
 - ❑ Standards for naming versions, e.g., version 1.0, 1.2, 2.0, etc.
 - ❑ Naming files, e.g., filename must be readable and start with 'PM' for project management related files
 - ❑ File structures, e.g., which files go to which folder

Concurrent development and testing

- Development and testing teams should agree on building process
- For example
 - Build completed at, say, 2pm
 - Deliver the build to testing team
 - Testing team starts testing on new build
 - Defects and issues are reported to dev. team

Frequent system building

- It is easier to fix problems early
 - Build and test early and often
- This encourages thorough unit testing
 - developers under pressure not to ‘break the build’
- Developers under pressure to progress
 - They cannot always claim “80% completion”
- Test early and test often
 - Shift-left testing

Configuration management planning

- Large projects generate a large number of work products
- All work products have to be managed
 - Specifications
 - Designs
 - Programs
 - Test data
 - User manuals
 - Etc.

CM plan

- CM plan defines
 - ❑ types of documents to be managed and a document naming scheme
 - ❑ who are in charge of CM activities
 - ❑ policies for change control and version management
 - ❑ CM tools and process for using tools
 - ❑ CM database (often included in tools)

Configuration item

■ Configuration item

- Anything under the configuration management
 - E.g., a source file, a document, a test case, a vision document

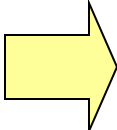
■ Configuration item needs to be identified

- Large projects typically produce thousands of documents
- Some of these documents must be maintained for the lifetime of the software
- Document naming scheme should be defined so that related documents have related names

Configuration database

- All CM information should be maintained in a configuration database
- CM DB allows queries about configurations
 - Who has a particular system version?
 - What platform is required for a particular version?
 - What versions are affected by a change to component X?
 - How many reported faults in version T?
- CM DB can be linked to the software being managed
- Examples
 - Github, Gitlab, Bitbucket

Topics covered

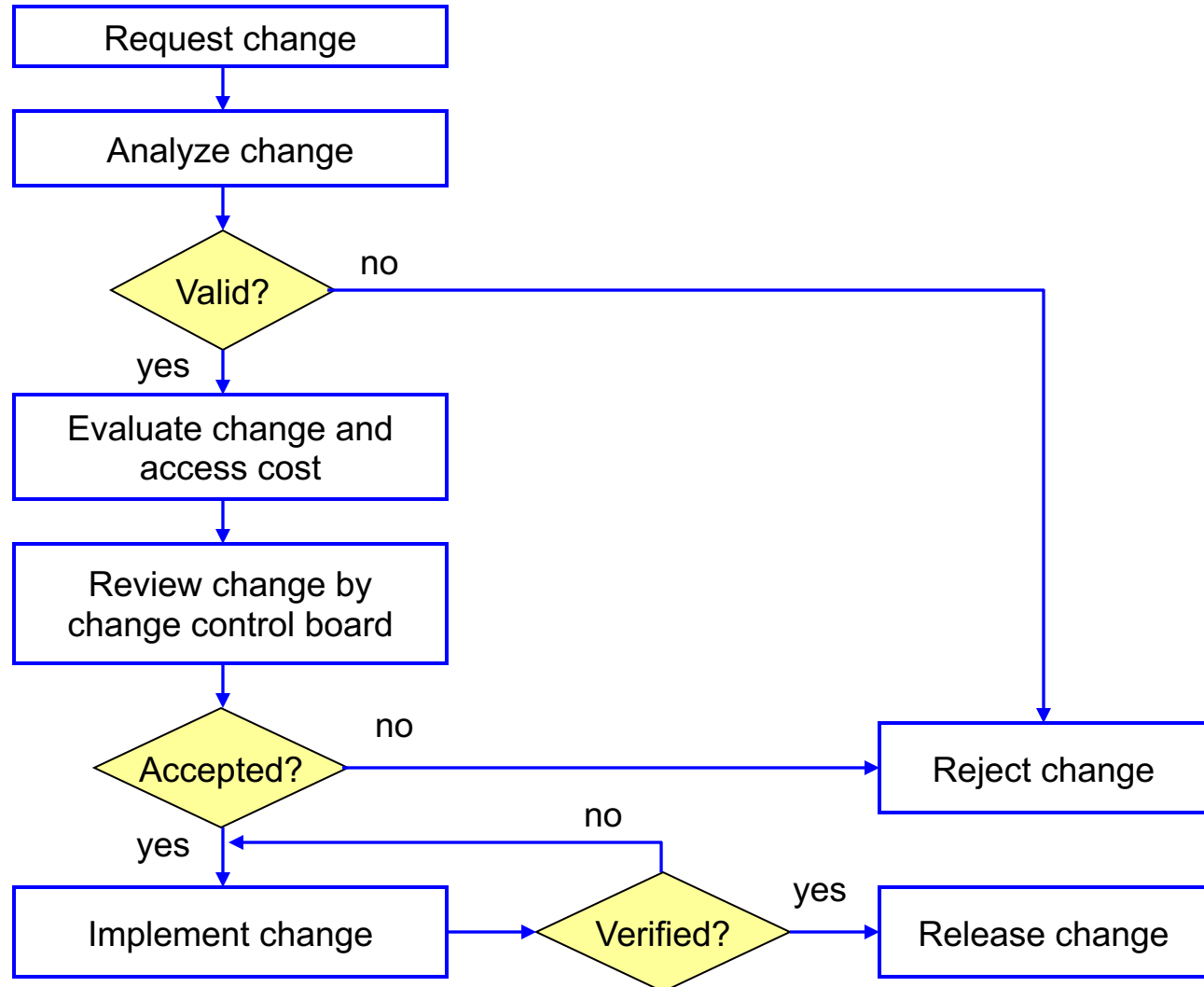
- 
- Configuration management planning
 - Change management
 - Version and release management
 - System building
 - CASE tools for configuration management

Change management

- Software systems are subject to continual change requests
 - From users
 - From developers
 - From market forces

- Change management
 - concerned with keeping track of changes
 - ensuring that changes are implemented effectively

A typical change management process



Change tracking tools

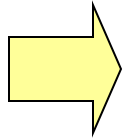
- A major problem in change management is tracking change status
- Change tracking tools keep track the status of each change request
- Many tools available, examples
 - Git, Github, Gitlab, Bitbucket, Azure DevOps Server, etc.

Derivation history

- Derivation history a record of changes in document or code
- It should record
 - ❑ change made
 - ❑ rationale for the change
 - ❑ who made the change
 - ❑ when it was implemented
- It may be included as a comment in code

Topics covered

- Configuration management planning
- Change management
- Version and release management
- System building
- CASE tools for configuration management



Version and release management

- Invent an identification scheme for system versions
- Plan when a new system version is to be produced
- Ensure that version management procedures and tools are properly applied
- Plan and distribute new system releases

Versions/variants/releases

■ Version

- ❑ An instance of a system which is **functionally** distinct in some way from other system instances

■ Variant

- ❑ An instance of a system which is **functionally identical** but non-functionally distinct from other instances

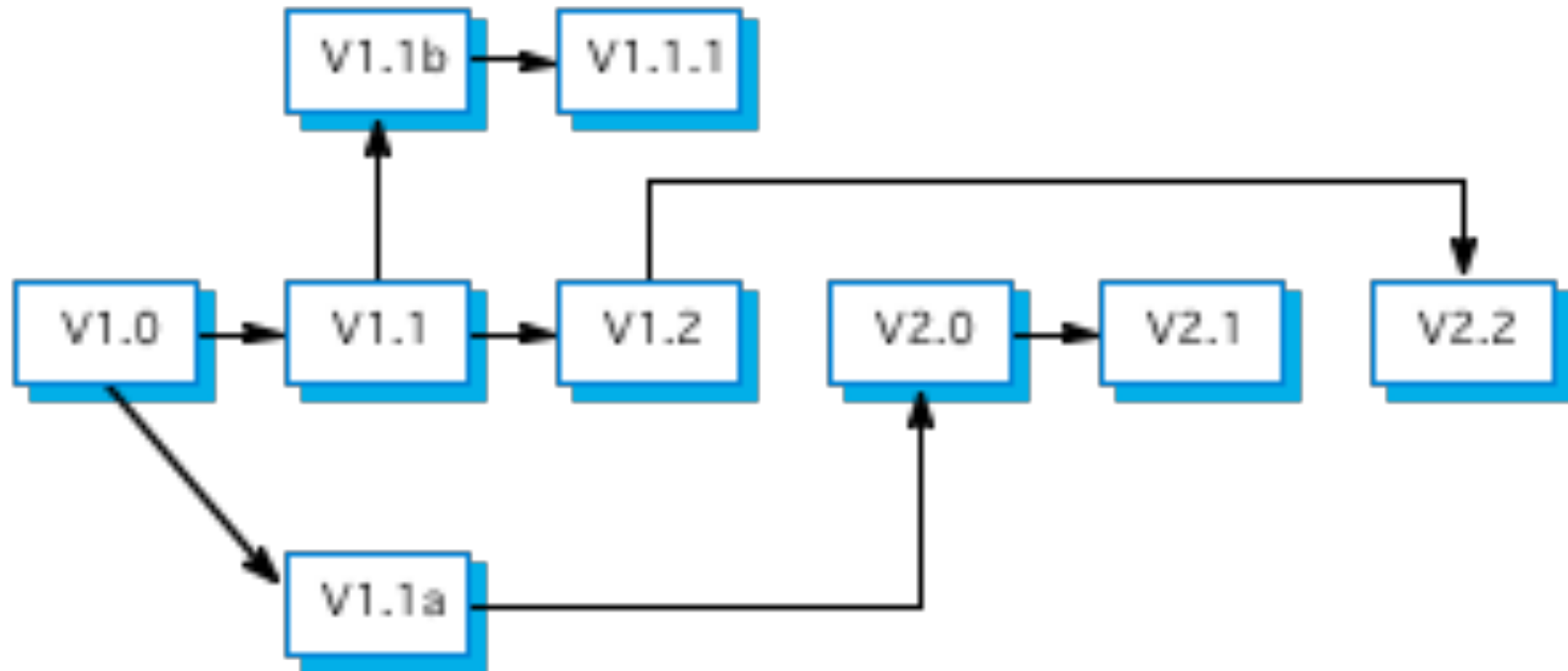
■ Release

- ❑ An instance of a system which is distributed to users outside of the development team
- ❑ External version becomes a release

Version numbering

- Version numbering is a common technique identifying software
- Simple naming scheme uses a linear derivation
 - V1, V1.1, V1.2, V2.1, V2.2 etc.
- Names are not meaningful

Version derivation structure



System releases

- Not just a set of executable programs
- May also include
 - ❑ Configuration files defining how the release is configured for a particular installation
 - ❑ Data files needed for system operation
 - ❑ An installation program or shell script to install the system on target hardware
 - ❑ Electronic and paper documentation
 - ❑ Packaging and associated publicity
- Systems are now normally released as downloadable installation files from the web

Release creation

- Release creation involves collecting all files and documentation required to create a system release
- Release notes
 - Common document for a new release
 - Describing what changes made, new features, software and hardware requirements, etc.

Create a release in Github

- Super simple
 - ❑ Create a tag
 - ❑ Specify release title
 - ❑ Upload binaries (if any)

The screenshot shows the 'Releases' tab of a GitHub repository. At the top, there are two tabs: 'Releases' (active) and 'Tags'. Below the tabs, there are two dropdown menus: 'Choose a tag' and 'Target: master'. A note below these says 'Choose an existing tag, or create a new tag on publish'. The main form has a 'Release title' input field. Below that are two tabs: 'Write' (active) and 'Preview'. The 'Write' tab contains a rich text editor with various formatting icons (H, B, I, list, code, link, table, quote, checkbox, @, link, undo). Below the editor is a large text area labeled 'Describe this release'. At the bottom of the form, there is a dashed box with a downward arrow and the text 'Attach binaries by dropping them'. Below this is a checkbox labeled 'This is a pre-release' with the subtext 'We'll point out that this release is identified as non-production ready.' At the very bottom are two buttons: 'Publish release' (green) and 'Save draft' (grey).

Releases Tags

Choose a tag Target: master

Choose an existing tag, or create a new tag on publish

Release title

Write Preview

H B I list code link table quote checkbox @ link undo

Describe this release

Attach files by dragging & dropping, selecting or pasting them.

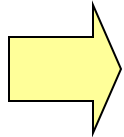
↓ Attach binaries by dropping them

☐ This is a pre-release
We'll point out that this release is identified as non-production ready.

Publish release Save draft

Topics covered

- Configuration management planning
- Change management
- Version and release management
- System building
- CASE tools for configuration management



System building

- Process of compiling and linking software components into an executable system
- This process is often supported by **automated tools** driven by build scripts or build code
 - CI/CD, DevOps tools

CM workbenches

■ Open workbenches

- ❑ Tools for each stage in the CM process are integrated through organizational procedures and scripts
- ❑ Gives flexibility in tool selection
- ❑ Examples
 - ❑ make, Apache Ant, Maven, CMake, Jenkins

■ Integrated workbenches

- ❑ Provide whole-process, integrated support for configuration management
- ❑ More tightly integrated tools so easier to use
- ❑ However, the cost is less flexibility in the tools used
- ❑ Examples
 - ❑ Azure DevOps, CircleCI, GitHub, TeamCity

Change and version management tools

- Version and release identification
 - Systems assign identifiers automatically
- Storage management
 - Storing differences between versions
- Change history recording
 - Record reasons for version creation
- Independent development
 - Only one version at a time may be checked out for change
 - Parallel working on different versions
- CI/CD and DevOps
 - Support the whole process of development and operation

Popular tools

■ Version control

- Git, GitHub, GitLab, Bitbucket, Subversion/SVN

■ Build creation

- Make, Apache Ant, BitBake, CMake, Buck

■ Continuous integration

- Commercial: CircleCI, TeamCity, Azure DevOps, Bitbucket, Github, Gitlab
- Open-source: Jenkins, Travis CI, Spinnaker, GitLab, Maven